

Notice, “length” is *not* a method, it is a property. You can use an “Initializer List” enclosed in braces to quickly insert values in your array:

```
int n[] = { 12, 44, 98, 1, 28 };
```

Because this list has five values in it, the resulting array will have five elements. If we had a char array, it would be like this:

```
char m[] = { 't', 'I', 'M', 'Y' };
```

6.1 Constant variables and the final qualifier

When used in a variable’s declaration, the keyword “final” indicates that the variable can never be changed.

```
final int ARRAY_SIZE = 10;
```

If you have chosen to make your variable “final” then you must initialize it in the same statement that declares it—you have no other alternative.

When we call a method and pass the method a primitive-data-type variable, we are always passing a copy of the original, not the original. Thus, if the method changes the variable it receives, only the copy is changed—not the original.

```
int x = 3;
```

x is a primitive variable. If we call a method with x as an argument, we make a copy of x and the original copy is not affected by any changes we make in the copy.

```
int x = 3;
yMethd( x )
public int yMethd( int y )
{
    return y++;
}
```

If we pass x to a method, a copy y is made and any changes happen to the copy. Now, we have `char[] out = { 'H', 'e', 'l', 'l', 'o' };`

If we pass the reference out to a method, we again pass a copy, but the copy points back to the original, and so our method can change the original.

6.2 Passing an array to a Method

As you saw in the previous example, when we wish to pass an array to a method, the array’s signature must be expecting to receive an array. Thus, the method was declared: